

Smart Travel Companion System with Budget Optimization using Recommendation

Project Code:

Frontend: java, Xml,Flutter , Android, dart

Restaurant.dart:

```
class Restaurant {
  final String id;
  final String name;
  final String city;
  final String cuisineType; // Indian, Chinese, Italian, etc.
  final double costForTwo;
  final double? rating;
  final int totalReviews;
  final String? imageUrl;
  final String? address;
  final double? distanceFromCityCenter; // in km
  final String? timings;
  final List<String>? specialties;

  Restaurant({
    required this.id,
    required this.name,
    required this.city,
    required this.cuisineType,
    required this.costForTwo,
    this.rating,
    this.totalReviews = 0,
    this.imageUrl,
    this.address,
    this.distanceFromCityCenter,
    this.timings,
    this.specialties,
  });

  factory Restaurant.fromJson(Map<String, dynamic> json) {
    return Restaurant(
      id: json['id'] as String,
      name: json['name'] as String,
      city: json['city'] as String,
      cuisineType: json['cuisineType'] as String,
      costForTwo: (json['costForTwo'] as num).toDouble(),
      rating: json['rating'] != null ? (json['rating'] as num).toDouble() : null,
      totalReviews: json['totalReviews'] as int? ?? 0,
      imageUrl: json['imageUrl'] as String?,
    );
  }
}
```

```

        address: json['address'] as String?,
        distanceFromCityCenter:
            json['distanceFromCityCenter'] != null
                ? (json['distanceFromCityCenter'] as num).toDouble()
                : null,
        timings: json['timings'] as String?,
        specialties: json['specialties'] != null
            ? (json['specialties'] as List<dynamic>).map((e) => e as String).toList()
            : null,
    );
}

Map<String, dynamic> toJson() {
    return {
        'id': id,
        'name': name,
        'city': city,
        'cuisineType': cuisineType,
        'costForTwo': costForTwo,
        'rating': rating,
        'totalReviews': totalReviews,
        'imageUrl': imageUrl,
        'address': address,
        'distanceFromCityCenter': distanceFromCityCenter,
        'timings': timings,
        'specialties': specialties,
    };
}
}

```

Place.dart:

```

class Place {
    final String id;
    final String name;
    final String city;
    final String category; // Monument, Park, Museum, Market, Adventure, etc.
    final double? entryFee;
    final String? timings;
    final double? rating;
    final int totalReviews;
    final String? imageUrl;
    final String? description;
    final double? visitDurationHours;
}

```

```

final double? latitude;
final double? longitude;
final String? address;

Place({
  required this.id,
  required this.name,
  required this.city,
  required this.category,
  this.entryFee,
  this.timings,
  this.rating,
  this.totalReviews = 0,
  this.imageUrl,
  this.description,
  this.visitDurationHours,
  this.latitude,
  this.longitude,
  this.address,
});

factory Place.fromJson(Map<String, dynamic> json) {
  return Place(
    id: json['id'] as String,
    name: json['name'] as String,
    city: json['city'] as String,
    category: json['category'] as String,
    entryFee: json['entryFee'] != null ? (json['entryFee'] as num).toDouble() : null,
    timings: json['timings'] as String?,
    rating: json['rating'] != null ? (json['rating'] as num).toDouble() : null,
    totalReviews: json['totalReviews'] as int? ?? 0,
    imageUrl: json['imageUrl'] as String?,
    description: json['description'] as String?,
    visitDurationHours: json['visitDurationHours'] != null
      ? (json['visitDurationHours'] as num).toDouble()
      : null,
    latitude: json['latitude'] != null ? (json['latitude'] as num).toDouble() : null,
    longitude: json['longitude'] != null ? (json['longitude'] as num).toDouble() :
null,
    address: json['address'] as String?,
  );
}

Map<String, dynamic> toJson() {
  return {
    'id': id,
    'name': name,
    'city': city,
    'category': category,

```

```

        'entryFee': entryFee,
        'timings': timings,
        'rating': rating,
        'totalReviews': totalReviews,
        'imageUrl': imageUrl,
        'description': description,
        'visitDurationHours': visitDurationHours,
        'latitude': latitude,
        'longitude': longitude,
        'address': address,
    };
}

double getTotalEntryFee(int numberOfPeople) {
    return (entryFee ?? 0) * numberOfPeople;
}
}

```

Hotel.dart:

```

class Hotel {
    final String id;
    final String name;
    final String city;
    final double pricePerNight;
    final double? rating;
    final int totalReviews;
    final List<String> amenities;
    final String? imageUrl;
    final String address;
    final double distanceFromCityCenter; // in km
    final int availableRooms;
    final String? description;

    Hotel({
        required this.id,
        required this.name,
        required this.city,
        required this.pricePerNight,
        this.rating,
        this.totalReviews = 0,
        this.amenities = const [],
        this.imageUrl,
        required this.address,
        this.distanceFromCityCenter = 0.0,
    });
}

```

```

        this.availableRooms = 0,
        this.description,
    });

    factory Hotel.fromJson(Map<String, dynamic> json) {
        return Hotel(
            id: json['id'] as String,
            name: json['name'] as String,
            city: json['city'] as String,
            pricePerNight: (json['pricePerNight'] as num).toDouble(),
            rating: json['rating'] != null ? (json['rating'] as num).toDouble() : null,
            totalReviews: json['totalReviews'] as int? ?? 0,
            amenities: (json['amenities'] as List<dynamic>?)
                ?.map((e) => e as String)
                .toList() ??
                [],
            imageUrl: json['imageUrl'] as String?,
            address: json['address'] as String,
            distanceFromCityCenter:
                (json['distanceFromCityCenter'] as num?)?.toDouble() ?? 0.0,
            availableRooms: json['availableRooms'] as int? ?? 0,
            description: json['description'] as String?,
        );
    }

    Map<String, dynamic> toJson() {
        return {
            'id': id,
            'name': name,
            'city': city,
            'pricePerNight': pricePerNight,
            'rating': rating,
            'totalReviews': totalReviews,
            'amenities': amenities,
            'imageUrl': imageUrl,
            'address': address,
            'distanceFromCityCenter': distanceFromCityCenter,
            'availableRooms': availableRooms,
            'description': description,
        };
    }

    double getTotalPrice(int nights, int rooms) {
        return pricePerNight * nights * rooms;
    }
}

```

Expense.dart:

```
class Expense {
  final String id;
  final String category; // Transport, Hotel, Food, Activities, etc.
  final String description;
  final double amount;
  final DateTime date;

  Expense({
    required this.id,
    required this.category,
    required this.description,
    required this.amount,
    required this.date,
  });

  factory Expense.fromJson(Map<String, dynamic> json) {
    return Expense(
      id: json['id'] as String,
      category: json['category'] as String,
      description: json['description'] as String,
      amount: (json['amount'] as num).toDouble(),
      date: DateTime.parse(json['date'] as String),
    );
  }

  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'category': category,
      'description': description,
      'amount': amount,
      'date': date.toIso8601String(),
    };
  }
}
```

City.dart:

```
class City {
  final String id;
  final String name;
```

```

final String state;
final double latitude;
final double longitude;
final String? imageUrl;

City({
  required this.id,
  required this.name,
  required this.state,
  required this.latitude,
  required this.longitude,
  this.imageUrl,
});

factory City.fromJson(Map<String, dynamic> json) {
  return City(
    id: json['id'] as String,
    name: json['name'] as String,
    state: json['state'] as String,
    latitude: (json['latitude'] as num).toDouble(),
    longitude: (json['longitude'] as num).toDouble(),
    imageUrl: json['imageUrl'] as String?,
  );
}

Map<String, dynamic> toJson() {
  return {
    'id': id,
    'name': name,
    'state': state,
    'latitude': latitude,
    'longitude': longitude,
    'imageUrl': imageUrl,
  };
}
}

```

Booking.dart:

```

import 'transport.dart';
import 'hotel.dart';
import 'place.dart';
import 'restaurant.dart';

enum BookingStatus {
  pending,
  confirmed,
}

```

```

    cancelled,
    completed,
}

class Booking {
    final String id;
    final String userId;
    final String fromCity;
    final String toCity;
    final DateTime tripDate;
    final int numberOfTravelers;
    final int numberOfDays;
    final Transport? selectedTransport;
    final Hotel? selectedHotel;
    final List<Place> selectedPlaces;
    final List<Restaurant> selectedRestaurants;
    final double totalCost;
    final double budget;
    final BookingStatus status;
    final DateTime bookingDate;
    final String? paymentId;
    final String? transactionId;

    Booking({
        required this.id,
        required this.userId,
        required this.fromCity,
        required this.toCity,
        required this.tripDate,
        required this.numberOfTravelers,
        required this.numberOfDays,
        this.selectedTransport,
        this.selectedHotel,
        this.selectedPlaces = const [],
        this.selectedRestaurants = const [],
        required this.totalCost,
        required this.budget,
        this.status = BookingStatus.pending,
        required this.bookingDate,
        this.paymentId,
        this.transactionId,
    });

    factory Booking.fromJson(Map<String, dynamic> json) {
        return Booking(
            id: json['id'] as String,
            userId: json['userId'] as String,
            fromCity: json['fromCity'] as String,
            toCity: json['toCity'] as String,

```

```

tripDate: DateTime.parse(json['tripDate'] as String),
numberOfTravelers: json['numberOfTravelers'] as int,
numberOfDays: json['numberOfDays'] as int,
selectedTransport: json['selectedTransport'] != null
    ? Transport.fromJson(json['selectedTransport'] as Map<String, dynamic>)
    : null,
selectedHotel: json['selectedHotel'] != null
    ? Hotel.fromJson(json['selectedHotel'] as Map<String, dynamic>)
    : null,
selectedPlaces: (json['selectedPlaces'] as List<dynamic>?)
    ?.map((e) => Place.fromJson(e as Map<String, dynamic>))
    .toList() ??
    [],
selectedRestaurants: (json['selectedRestaurants'] as List<dynamic>?)
    ?.map((e) => Restaurant.fromJson(e as Map<String, dynamic>))
    .toList() ??
    [],
totalCost: (json['totalCost'] as num).toDouble(),
budget: (json['budget'] as num).toDouble(),
status: BookingStatus.values.firstWhere(
    (e) => e.toString().split('.').last == json['status'],
    orElse: () => BookingStatus.pending,
),
bookingDate: DateTime.parse(json['bookingDate'] as String),
paymentId: json['paymentId'] as String?,
transactionId: json['transactionId'] as String?,
);
}

```

```

Map<String, dynamic> toJson() {
    return {
        'id': id,
        'userId': userId,
        'fromCity': fromCity,
        'toCity': toCity,
        'tripDate': tripDate.toIso8601String(),
        'numberOfTravelers': numberOfTravelers,
        'numberOfDays': numberOfDays,
        'selectedTransport': selectedTransport?.toJson(),
        'selectedHotel': selectedHotel?.toJson(),
        'selectedPlaces': selectedPlaces.map((e) => e.toJson()).toList(),
        'selectedRestaurants': selectedRestaurants.map((e) => e.toJson()).toList(),
        'totalCost': totalCost,
        'budget': budget,
        'status': status.toString().split('.').last,
        'bookingDate': bookingDate.toIso8601String(),
        'paymentId': paymentId,
        'transactionId': transactionId,
    };
}

```

```
}  
}
```

In pages:

Home_page.dart:

```
import 'package:flutter/material.dart';  
import 'package:provider/provider.dart';  
import 'package:intl/intl.dart';  
import '../models/city.dart';  
import '../providers/trip_provider.dart';  
import '../services/dummy_data_service.dart';  
import 'transport_page.dart';  
  
class HomePage extends StatefulWidget {  
  const HomePage({super.key});  
  
  @override  
  State<HomePage> createState() => _HomePageState();  
}  
  
class _HomePageState extends State<HomePage> {  
  final _budgetController = TextEditingController();  
  final _travelersController = TextEditingController(text: '1');  
  final _daysController = TextEditingController(text: '1');  
  final DummyDataService _dataService = DummyDataService();  
  List<City> _cities = [];  
  DateTime? _selectedDate;  
  
  @override  
  void initState() {  
    super.initState();  
    _loadCities();  
  }  
  
  void _loadCities() {  
    setState(() {  
      _cities = _dataService.getCities();  
    });  
  }  
  
  @override  
  void dispose() {
```

```

    _budgetController.dispose();
    _travelersController.dispose();
    _daysController.dispose();
    super.dispose();
}

Future<void> _selectDate(BuildContext context) async {
    final DateTime? picked = await showDatePicker(
        context: context,
        initialDate: DateTime.now().add(const Duration(days: 1)),
        firstDate: DateTime.now(),
        lastDate: DateTime.now().add(const Duration(days: 365)),
    );
    if (picked != null) {
        setState(() {
            _selectedDate = picked;
        });
    }
}

void _planTrip(BuildContext context) {
    final tripProvider = Provider.of<TripProvider>(context, listen: false);

    final budget = double.tryParse(_budgetController.text);
    final travelers = int.tryParse(_travelersController.text) ?? 1;
    final days = int.tryParse(_daysController.text) ?? 1;

    if (budget == null || budget <= 0) {
        ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('Please enter a valid budget')),
        );
        return;
    }

    if (tripProvider.fromCity == null || tripProvider.toCity == null) {
        ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('Please select origin and destination cities')),
        );
        return;
    }

    tripProvider.setBudget(budget);
    tripProvider.setNumberOfTravelers(travelers);
    tripProvider.setNumberOfDays(days);
    tripProvider.setTripDate(_selectedDate);

    tripProvider.generateTripPlan().then((_) {
        if (tripProvider.errorMessage == null) {

```

```

        Navigator.push(
            context,
            MaterialPageRoute(builder: (context) => const TransportPage()),
        );
    } else {
        ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(content: Text(tripProvider.errorMessage!)),
        );
    }
});
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(
            title: const Text('Pocket Trip'),
            backgroundColor: Colors.blue.shade700,
            foregroundColor: Colors.white,
            elevation: 0,
        ),
        body: SingleChildScrollView(
            padding: const EdgeInsets.all(16.0),
            child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                children: [
                    // Header
                    Container(
                        padding: const EdgeInsets.all(20),
                        decoration: BoxDecoration(
                            gradient: LinearGradient(
                                colors: [Colors.blue.shade700, Colors.blue.shade400],
                                begin: Alignment.topLeft,
                                end: Alignment.bottomRight,
                            ),
                        ),
                        borderRadius: BorderRadius.circular(12),
                    ),
                    child: Column(
                        children: [
                            const Icon(Icons.flight_takeoff, size: 48, color: Colors.white),
                            const SizedBox(height: 8),
                            const Text(
                                'Plan Your Perfect Trip',
                                style: TextStyle(
                                    fontSize: 24,
                                    fontWeight: FontWeight.bold,
                                    color: Colors.white,
                                ),
                            ),
                        ],
                    ),
                ],
            ),
        ),
    );
}

```

```

        const SizedBox(height: 4),
        Text(
          'Discover India with smart budget planning',
          style: TextStyle(
            fontSize: 14,
            color: Colors.white.withOpacity(0.9),
          ),
        ),
      ],
    ),
  ),
const SizedBox(height: 24),

// City Selection
Consumer<TripProvider>(
  builder: (context, tripProvider, _) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        _buildCityDropdown(
          'From',
          tripProvider.fromCity,
          (city) => tripProvider.setFromCity(city),
        ),
        const SizedBox(height: 16),
        _buildCityDropdown(
          'To',
          tripProvider.toCity,
          (city) => tripProvider.setToCity(city),
        ),
      ],
    );
  },
),

const SizedBox(height: 24),

// Budget Input
TextField(
  controller: _budgetController,
  keyboardType: TextInputType.number,
  decoration: InputDecoration(
    labelText: 'Budget (₹)',
    hintText: 'Enter your budget',
    prefixIcon: const Icon(Icons.currency_rupee),
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(12),
    ),
  ),
  filled: true,

```

```

        fillColor: Colors.grey.shade50,
      ),
    ),

    const SizedBox(height: 16),

    // Number of Travelers
    TextField(
      controller: _travelersController,
      keyboardType: TextInputType.number,
      decoration: InputDecoration(
        labelText: 'Number of Travelers',
        hintText: 'Enter number of travelers',
        prefixIcon: const Icon(Icons.people),
        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(12),
        ),
        filled: true,
        fillColor: Colors.grey.shade50,
      ),
    ),

    const SizedBox(height: 16),

    // Number of Days
    TextField(
      controller: _daysController,
      keyboardType: TextInputType.number,
      decoration: InputDecoration(
        labelText: 'Number of Days',
        hintText: 'Enter number of days',
        prefixIcon: const Icon(Icons.calendar_today),
        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(12),
        ),
        filled: true,
        fillColor: Colors.grey.shade50,
      ),
    ),

    const SizedBox(height: 16),

    // Trip Date
    InkWell(
      onTap: () => _selectDate(context),
      child: Container(
        padding: const EdgeInsets.all(16),
        decoration: BoxDecoration(

```

```

        border: Border.all(color: Colors.grey.shade300),
        borderRadius: BorderRadius.circular(12),
        color: Colors.grey.shade50,
      ),
      child: Row(
        children: [
          const Icon(Icons.date_range),
          const SizedBox(width: 12),
          Text(
            _selectedDate == null
              ? 'Select Trip Date'
              : DateFormat('MMM dd, yyyy').format(_selectedDate!),
            style: TextStyle(
              fontSize: 16,
              color: _selectedDate == null
                ? Colors.grey.shade600
                : Colors.black,
            ),
          ),
        ],
      ),
    ),
  ),
),

const SizedBox(height: 32),

// Plan Trip Button
Consumer<TripProvider>(
  builder: (context, tripProvider, _) {
    return ElevatedButton(
      onPressed: tripProvider.isLoading ? null : () => _planTrip(context),
      style: ElevatedButton.styleFrom(
        backgroundColor: Colors.blue.shade700,
        foregroundColor: Colors.white,
        padding: const EdgeInsets.symmetric(vertical: 16),
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(12),
        ),
        elevation: 2,
      ),
      child: tripProvider.isLoading
        ? const SizedBox(
            height: 20,
            width: 20,
            child: CircularProgressIndicator(
              strokeWidth: 2,
              valueColor: AlwaysStoppedAnimation<Color>(Colors.white),
            ),
          )
        )
    )
  )
)

```

```

        : const Text(
          'Plan My Trip',
          style: TextStyle(
            fontSize: 18,
            fontWeight: FontWeight.bold,
          ),
        ),
      ),
    ],
  ),
),
);
}

Widget _buildCityDropdown(
  String label,
  City? selectedCity,
  Function(City) onChanged,
) {
  return DropdownButtonFormField<City>(
    value: selectedCity,
    decoration: InputDecoration(
      labelText: label,
      prefixIcon: Icon(label == 'From' ? Icons.location_on : Icons.location_city),
      border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(12),
      ),
      filled: true,
      fillColor: Colors.grey.shade50,
    ),
    items: _cities.map((City city) {
      return DropdownMenuItem<City>(
        value: city,
        child: Text('${city.name}, ${city.state}'),
      );
    }).toList(),
    onChanged: (City? city) {
      if (city != null) {
        onChanged(city);
      }
    },
  );
}
}

```

Hotels_page.dart:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../models/hotel.dart';
import '../providers/trip_provider.dart';
import 'places_page.dart';

class HotelsPage extends StatelessWidget {
  const HotelsPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Select Hotel'),
        backgroundColor: Colors.blue.shade700,
        foregroundColor: Colors.white,
      ),
      body: Consumer<TripProvider>(
        builder: (context, tripProvider, _) {
          final hotels = tripProvider.hotels;
          final selectedHotel = tripProvider.selectedHotel;
          final numberOfDays = tripProvider.numberOfDays;

          return Column(
            children: [
              // Info banner
              Container(
                padding: const EdgeInsets.all(16),
                color: Colors.blue.shade50,
                child: Row(
                  children: [
                    Icon(Icons.info_outline, color: Colors.blue.shade700),
                    const SizedBox(width: 8),
                    Expanded(
                      child: Text(
                        'Showing hotels for ${numberOfDays} night(s)',
                        style: TextStyle(
                          fontSize: 14,
                          color: Colors.blue.shade900,
                        ),
                      ),
                    ),
                  ],
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```



```

        ? ClipRRect(
          borderRadius: BorderRadius.circular(8),
          child: Image.network(
            hotel.imageUrl!,
            fit: BoxFit.cover,
          ),
        )
      : Icon(
        Icons.hotel,
        size: 40,
        color: Colors.grey.shade600,
      ),
    ),
    const SizedBox(width: 12),
    Expanded(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            hotel.name,
            style: const TextStyle(
              fontSize: 18,
              fontWeight: FontWeight.bold,
            ),
          ),
          const SizedBox(height: 4),
          Row(
            children: [
              Icon(
                Icons.location_on,
                size: 14,
                color: Colors.grey.shade600,
              ),
              Expanded(
                child: Text(
                  hotel.address,
                  style: TextStyle(
                    fontSize: 12,
                    color: Colors.grey.shade600,
                  ),
                ),
                maxLines: 1,
                overflow: TextOverflow.ellipsis,
              ),
            ],
          ),
        ],
      ),
    ),
    if (hotel.rating != null) ...[
      const SizedBox(height: 4),
      Row(
        children: [

```

```

        Icon(
          Icons.star,
          size: 16,
          color: Colors.amber,
        ),
        const SizedBox(width: 4),
        Text(
          '${hotel.rating!.toStringAsFixed(1)} (${hotel.totalReviews} reviews)',
          style: const TextStyle(
            fontSize: 14,
            fontWeight: FontWeight.w600,
          ),
        ),
      ],
    ),
  ],
),
),
Column(
  crossAxisAlignment: CrossAxisAlignment.end,
  children: [
    Text(
      '₹${totalPrice.toStringAsFixed(0)}',
      style: TextStyle(
        fontSize: 20,
        fontWeight: FontWeight.bold,
        color: Colors.blue.shade700,
      ),
    ),
    Text(
      '₹${hotel.pricePerNight.toStringAsFixed(0)}/night',
      style: TextStyle(
        fontSize: 12,
        color: Colors.grey.shade600,
      ),
    ),
  ],
),
],
),
if (hotel.amenities.isNotEmpty) ...[
  const SizedBox(height: 12),
  Wrap(
    spacing: 8,
    runSpacing: 8,
    children: hotel.amenities.take(4).map((amenity) {
      return Chip(

```

```

        label: Text(
          amenity,
          style: const TextStyle(fontSize: 12),
        ),
        backgroundColor: Colors.grey.shade100,
        padding: EdgeInsets.zero,
        materialTapTargetSize:
            MaterialTapTargetSize.shrinkWrap,
      );
    }).toList(),
  ],
  if (isSelected) ...[
    const SizedBox(height: 8),
    Container(
      padding: const EdgeInsets.symmetric(
        horizontal: 8,
        vertical: 4,
      ),
      decoration: BoxDecoration(
        color: Colors.blue.shade700,
        borderRadius: BorderRadius.circular(4),
      ),
      child: const Text(
        'SELECTED',
        style: TextStyle(
          color: Colors.white,
          fontSize: 12,
          fontWeight: FontWeight.bold,
        ),
      ),
    ),
  ],
],
),
);
},
),
// Continue button
Container(
padding: const EdgeInsets.all(16),
decoration: BoxDecoration(
color: Colors.white,
boxShadow: [
BoxShadow(
```



```

import '../models/booking.dart';
import '../services/payment_service.dart';
import '../services/booking_service.dart';
import 'package:provider/provider.dart';
import '../providers/trip_provider.dart';

class PaymentPage extends StatefulWidget {
  final Booking booking;

  const PaymentPage({super.key, required this.booking});

  @override
  State<PaymentPage> createState() => _PaymentPageState();
}

class _PaymentPageState extends State<PaymentPage> {
  final PaymentService _paymentService = PaymentService();
  bool _isProcessing = false;

  @override
  void initState() {
    super.initState();
    _paymentService.initialize();
  }

  @override
  void dispose() {
    _paymentService.dispose();
    super.dispose();
  }

  Future<void> _processPayment() async {
    setState(() {
      _isProcessing = true;
    });

    try {
      final response = await _paymentService.initiatePayment(
        amount: widget.booking.totalCost,
        orderId: widget.booking.id,
        name: 'User Name', // Get from user profile
        email: 'user@example.com', // Get from user profile
        phone: '9876543210', // Get from user profile
        description: 'Trip booking from ${widget.booking.fromCity} to
${widget.booking.toCity}',
      );
    }
  }

```

```

final bookingService = BookingService();

if (response.success) {
  // Update booking status
  await bookingService.updateBookingStatus(
    bookingId: widget.booking.id,
    status: BookingStatus.confirmed,
    paymentId: response.paymentId,
    transactionId: response.orderId,
  );

  if (mounted) {
    // Show success dialog
    showDialog(
      context: context,
      barrierDismissible: false,
      builder: (context) => AlertDialog(
        title: const Text('Payment Successful!'),
        content: Column(
          mainAxisAlignment: MainAxisAlignment.start,
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Text('Your trip has been booked successfully.'),
            const SizedBox(height: 8),
            Text('Payment ID: ${response.paymentId ?? "N/A"}'),
            Text('Transaction ID: ${response.orderId ?? "N/A"}'),
          ],
        ),
        actions: [
          TextButton(
            onPressed: () {
              Navigator.of(context).popUntil((route) => route.isFirst);
              // Reset trip provider
              Provider.of<TripProvider>(context, listen: false).reset();
            },
            child: const Text('OK'),
          ),
        ],
      ),
    );
  }
} else {
  if (mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text('Payment failed: ${response.error ?? "Unknown error"}'),
        backgroundColor: Colors.red,
      ),
    );
  }
}

```

```

    }
  }
} catch (e) {
  if (mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text('Error: ${e.toString()}'),
        backgroundColor: Colors.red,
      ),
    );
  }
} finally {
  if (mounted) {
    setState(() {
      _isProcessing = false;
    });
  }
}
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Payment'),
      backgroundColor: Colors.blue.shade700,
      foregroundColor: Colors.white,
    ),
    body: SingleChildScrollView(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // Booking Summary
          Card(
            child: Padding(
              padding: const EdgeInsets.all(16),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  const Text(
                    'Booking Summary',
                    style: TextStyle(
                      fontSize: 20,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                  const SizedBox(height: 16),
                  _buildSummaryRow('From', widget.booking.fromCity),
                ],
              ),
            ),
          ),
        ],
      ),
    ),
  );
}

```

```

        _buildSummaryRow('To', widget.booking.toCity),
        _buildSummaryRow('Travelers', '${widget.booking.numberOfTravelers}'),
        _buildSummaryRow('Duration', '${widget.booking.numberOfDays}
day(s)'),

        const Divider(),
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            const Text(
              'Total Amount',
              style: TextStyle(
                fontSize: 20,
                fontWeight: FontWeight.bold,
              ),
            ),
            Text(
              '₹${widget.booking.totalCost.toStringAsFixed(0)}',
              style: TextStyle(
                fontSize: 24,
                fontWeight: FontWeight.bold,
                color: Colors.blue.shade700,
              ),
            ),
          ],
        ),
      ],
    ),
  ],
),
),

const SizedBox(height: 24),

// Payment Methods
const Text(
  'Select Payment Method',
  style: TextStyle(
    fontSize: 18,
    fontWeight: FontWeight.bold,
  ),
),
const SizedBox(height: 16),

// UPI
_buildPaymentMethodCard(
  icon: Icons.account_balance_wallet,
  title: 'UPI',
  subtitle: 'Pay using UPI (GPay, PhonePe, Paytm)',
  onTap: _processPayment,
),

```

```

const SizedBox(height: 12),

// Cards
_buildPaymentMethodCard(
  icon: Icons.credit_card,
  title: 'Debit/Credit Card',
  subtitle: 'Visa, Mastercard, RuPay',
  onTap: _processPayment,
),

const SizedBox(height: 12),

// Net Banking
_buildPaymentMethodCard(
  icon: Icons.account_balance,
  title: 'Net Banking',
  subtitle: 'All major banks',
  onTap: _processPayment,
),

const SizedBox(height: 32),

// Pay Now Button
SizedBox(
  width: double.infinity,
  child: ElevatedButton(
    onPressed: _isProcessing ? null : _processPayment,
    style: ElevatedButton.styleFrom(
      backgroundColor: Colors.blue.shade700,
      foregroundColor: Colors.white,
      padding: const EdgeInsets.symmetric(vertical: 16),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12),
      ),
    ),
    child: _isProcessing
      ? const SizedBox(
          height: 20,
          width: 20,
          child: CircularProgressIndicator(
            strokeWidth: 2,
            valueColor: AlwaysStoppedAnimation<Color>(Colors.white),
          ),
        )
      : const Text(
          'Pay Now',
          style: TextStyle(

```

```

        fontSize: 18,
        fontWeight: FontWeight.bold,
      ),
    ),
  ),
),

const SizedBox(height: 16),

// Web platform notice
if (kIsWeb)
  Container(
    padding: const EdgeInsets.all(12),
    margin: const EdgeInsets.only(bottom: 12),
    decoration: BoxDecoration(
      color: Colors.orange.shade50,
      borderRadius: BorderRadius.circular(8),
      border: Border.all(color: Colors.orange.shade200),
    ),
    child: Row(
      children: [
        Icon(Icons.info_outline, color: Colors.orange.shade700, size: 20),
        const SizedBox(width: 8),
        Expanded(
          child: Text(
            'Note: Payment simulation on web. Real payments work on
Android/iOS apps.',
            style: TextStyle(
              fontSize: 12,
              color: Colors.orange.shade900,
            ),
          ),
        ),
      ],
    ),
  ),

// Security note
Container(
  padding: const EdgeInsets.all(12),
  decoration: BoxDecoration(
    color: Colors.green.shade50,
    borderRadius: BorderRadius.circular(8),
    border: Border.all(color: Colors.green.shade200),
  ),
  child: Row(
    children: [
      Icon(Icons.lock, color: Colors.green.shade700, size: 20),
      const SizedBox(width: 8),

```

```

        Expanded(
          child: Text(
            'Your payment is secured with 256-bit SSL encryption',
            style: TextStyle(
              fontSize: 12,
              color: Colors.green.shade900,
            ),
          ),
        ),
      ],
    ),
  ),
],
),
);
}

```

```

Widget _buildSummaryRow(String label, String value) {
  return Padding(
    padding: const EdgeInsets.only(bottom: 8),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.spaceBetween,
      children: [
        Text(
          label,
          style: TextStyle(
            fontSize: 14,
            color: Colors.grey.shade700,
          ),
        ),
        Text(
          value,
          style: const TextStyle(
            fontSize: 14,
            fontWeight: FontWeight.w600,
          ),
        ),
      ],
    ),
  );
}

```

```

Widget _buildPaymentMethodCard({
  required IconData icon,
  required String title,
  required String subtitle,
  required VoidCallback onTap,
}) {

```

```

return Card(
  child: InkWell(
    onTap: onTap,
    borderRadius: BorderRadius.circular(12),
    child: Padding(
      padding: const EdgeInsets.all(16),
      child: Row(
        children: [
          Container(
            padding: const EdgeInsets.all(12),
            decoration: BoxDecoration(
              color: Colors.blue.shade50,
              borderRadius: BorderRadius.circular(8),
            ),
            child: Icon(icon, color: Colors.blue.shade700),
          ),
          const SizedBox(width: 16),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  title,
                  style: const TextStyle(
                    fontSize: 16,
                    fontWeight: FontWeight.w600,
                  ),
                ),
                Text(
                  subtitle,
                  style: TextStyle(
                    fontSize: 12,
                    color: Colors.grey.shade600,
                  ),
                ),
              ],
            ),
          ),
          Icon(Icons.chevron_right, color: Colors.grey.shade400),
        ],
      ),
    ),
  );
}

```

Place_page.dart:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../models/place.dart';
import '../providers/trip_provider.dart';
import 'summary_page.dart';

class PlacesPage extends StatelessWidget {
  const PlacesPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Select Places to Visit'),
        backgroundColor: Colors.blue.shade700,
        foregroundColor: Colors.white,
      ),
      body: Consumer<TripProvider>(
        builder: (context, tripProvider, _) {
          final places = tripProvider.places;
          final selectedPlaces = tripProvider.selectedPlaces;
          final numberOfTravelers = tripProvider.numberOfTravelers;

          return Column(
            children: [
              // Info banner
              Container(
                padding: const EdgeInsets.all(16),
                color: Colors.blue.shade50,
                child: Row(
                  children: [
                    Icon(Icons.info_outline, color: Colors.blue.shade700),
                    const SizedBox(width: 8),
                    Expanded(
                      child: Text(
                        'Select places you want to visit (${selectedPlaces.length}
selected)',
                        style: TextStyle(
                          fontSize: 14,
                          color: Colors.blue.shade900,
                        ),
                      ),
                    ),
                  ],
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

```

// Places list
Expanded(
  child: places.isEmpty
    ? const Center(
        child: Text('No places available'),
      )
    : ListView.builder(
        padding: const EdgeInsets.all(16),
        itemCount: places.length,
        itemBuilder: (context, index) {
          final place = places[index];
          final isSelected = selectedPlaces.any((p) => p.id == place.id);
          final totalEntryFee =
place.getTotalEntryFee(numberOfTravelers);

          return Card(
            margin: const EdgeInsets.only(bottom: 12),
            elevation: isSelected ? 4 : 1,
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(12),
              side: BorderSide(
                color: isSelected
                  ? Colors.blue.shade700
                  : Colors.transparent,
                width: 2,
              ),
            ),
            child: InkWell(
              onTap: () {
                tripProvider.togglePlace(place);
              },
              borderRadius: BorderRadius.circular(12),
              child: Padding(
                padding: const EdgeInsets.all(16),
                child: Row(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    // Place icon
                    Container(
                      width: 60,
                      height: 60,
                      decoration: BoxDecoration(
                        color: Colors.blue.shade100,
                        borderRadius: BorderRadius.circular(8),
                      ),
                    ),
                    Icon(
                      _getPlaceIcon(place.category),
                      color: Colors.blue.shade700,
                      size: 30,

```

```

    ),
  ),
  const SizedBox(width: 12),
  Expanded(
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(
          place.name,
          style: const TextStyle(
            fontSize: 18,
            fontWeight: FontWeight.bold,
          ),
        ),
        const SizedBox(height: 4),
        Chip(
          label: Text(
            place.category,
            style: const TextStyle(fontSize: 11),
          ),
          backgroundColor: Colors.grey.shade200,
          padding: EdgeInsets.zero,
          materialTapTargetSize:
            MaterialTapTargetSize.shrinkWrap,
        ),
        if (place.rating != null) ...[
          const SizedBox(height: 4),
          Row(
            children: [
              Icon(
                Icons.star,
                size: 16,
                color: Colors.amber,
              ),
              const SizedBox(width: 4),
              Text(
                '${place.rating!.toStringAsFixed(1)}
                ( ${place.totalReviews} reviews)',
                style: const TextStyle(
                  fontSize: 12,
                  fontWeight: FontWeight.w600,
                ),
              ),
            ],
          ),
        ],
        if (place.timings != null) ...[
          const SizedBox(height: 4),
          Row(
            children: [

```



```

        ),
        if (isSelected)
            const Icon(
                Icons.check_circle,
                color: Colors.blue,
                size: 24,
            ),
    ],
),
],
),
),
),
),
);
},
),
),

// Continue button
Container(
  padding: const EdgeInsets.all(16),
  decoration: BoxDecoration(
    color: Colors.white,
    boxShadow: [
      BoxShadow(
        color: Colors.black.withOpacity(0.1),
        blurRadius: 4,
        offset: const Offset(0, -2),
      ),
    ],
  ),
  child: SafeArea(
    child: ElevatedButton(
      onPressed: () {
        Navigator.push(
          context,
          MaterialPageRoute(
            builder: (context) => const SummaryPage(),
          ),
        );
      },
      style: ElevatedButton.styleFrom(
        backgroundColor: Colors.blue.shade700,
        foregroundColor: Colors.white,
        padding: const EdgeInsets.symmetric(vertical: 16),
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(12),
        ),
      ),
    ),
  ),

```

```

        child: const Text(
          'View Summary',
          style: TextStyle(
            fontSize: 18,
            fontWeight: FontWeight.bold,
          ),
        ),
      ),
    ),
  ],
);
},
),
);
}

IconData _getPlaceIcon(String category) {
  switch (category.toLowerCase()) {
    case 'monument':
      return Icons.account_balance;
    case 'park':
      return Icons.park;
    case 'museum':
      return Icons.museum;
    case 'market':
      return Icons.shopping_bag;
    case 'beach':
      return Icons.beach_access;
    case 'adventure':
      return Icons.terrain;
    default:
      return Icons.place;
  }
}
}

```

Summary_page.dart:

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:intl/intl.dart';
import '../providers/trip_provider.dart';
import '../services/booking_service.dart';
import '../models/booking.dart';
import '../models/transport.dart';

```

```

import 'payment_page.dart';

class SummaryPage extends StatelessWidget {
  const SummaryPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Trip Summary'),
        backgroundColor: Colors.blue.shade700,
        foregroundColor: Colors.white,
      ),
      body: Consumer<TripProvider>(
        builder: (context, tripProvider, _) {
          final totalCost = tripProvider.getTotalCost();
          final budget = tripProvider.budget;
          final isWithinBudget = totalCost <= budget;

          return SingleChildScrollView(
            padding: const EdgeInsets.all(16),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                // Trip Overview
                Card(
                  child: Padding(
                    padding: const EdgeInsets.all(16),
                    child: Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: [
                        const Text(
                          'Trip Overview',
                          style: TextStyle(
                            fontSize: 20,
                            fontWeight: FontWeight.bold,
                          ),
                        ),
                        const SizedBox(height: 16),
                        _buildSummaryRow(
                          'From',
                          '${tripProvider.fromCity?.name ?? "N/A"}',
                          Icons.location_on,
                        ),
                        _buildSummaryRow(
                          'To',
                          '${tripProvider.toCity?.name ?? "N/A"}',
                          Icons.location_city,
                        ),
                      ],
                    ),
                  ),
                ),
              ],
            ),
          );
        },
      ),
    );
  }
}

```

```

        _buildSummaryRow(
          'Travelers',
          '${tripProvider.numberOfTravelers}',
          Icons.people,
        ),
        _buildSummaryRow(
          'Duration',
          '${tripProvider.numberOfDays} day(s)',
          Icons.calendar_today,
        ),
        if (tripProvider.tripDate != null)
          _buildSummaryRow(
            'Trip Date',
            DateFormat('MMM dd, yyyy').format(tripProvider.tripDate!),
            Icons.date_range,
          ),
      ],
    ),
  ),
),

const SizedBox(height: 16),

// Transport
if (tripProvider.selectedTransport != null)
  Card(
    child: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              Text(
                tripProvider.selectedTransport!.type.emoji,
                style: const TextStyle(fontSize: 24),
              ),
              const SizedBox(width: 8),
              const Text(
                'Transport',
                style: TextStyle(
                  fontSize: 18,
                  fontWeight: FontWeight.bold,
                ),
              ),
            ],
          ),
          const SizedBox(height: 8),
          Text(

```

```

        '${tripProvider.selectedTransport!.type.displayName} -
        ${tripProvider.selectedTransport!.operatorName ?? "N/A"}',
        style: TextStyle(
          fontSize: 14,
          color: Colors.grey.shade700,
        ),
      ),
      const SizedBox(height: 4),
      Text(
        '₹${tripProvider.selectedTransport!.getPriceForPassengers(tripProvider.numberOfTravelers)
        .toStringAsFixed(0)}',
        style: TextStyle(
          fontSize: 16,
          fontWeight: FontWeight.w600,
          color: Colors.blue.shade700,
        ),
      ),
    ],
  ),
),
),

const SizedBox(height: 16),

// Hotel
if (tripProvider.selectedHotel != null)
  Card(
    child: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const Row(
            children: [
              Icon(Icons.hotel, size: 24),
              SizedBox(width: 8),
              Text(
                'Hotel',
                style: TextStyle(
                  fontSize: 18,
                  fontWeight: FontWeight.bold,
                ),
              ),
            ],
          ),
          const SizedBox(height: 8),
          Text(
            tripProvider.selectedHotel!.name,

```

```

        style: TextStyle(
          fontSize: 14,
          color: Colors.grey.shade700,
        ),
      ),
      const SizedBox(height: 4),
      Text(
        '₹${tripProvider.selectedHotel!.getTotalPrice(tripProvider.numberOfDays,
1).toStringAsFixed(0)} for ${tripProvider.numberOfDays} night(s)',
        style: TextStyle(
          fontSize: 16,
          fontWeight: FontWeight.w600,
          color: Colors.blue.shade700,
        ),
      ),
    ],
  ),
),
),
),

const SizedBox(height: 16),

// Places
if (tripProvider.selectedPlaces.isNotEmpty)
  Card(
    child: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const Row(
            children: [
              Icon(Icons.place, size: 24),
              SizedBox(width: 8),
              Text(
                'Places to Visit',
                style: TextStyle(
                  fontSize: 18,
                  fontWeight: FontWeight.bold,
                ),
              ),
            ],
          ),
          const SizedBox(height: 8),
          ...tripProvider.selectedPlaces.map((place) {
            final entryFee =
place.getTotalEntryFee(tripProvider.numberOfTravelers);
            return Padding(

```

```
padding: const EdgeInsets.only(bottom: 8),
child: Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  children: [
    Expanded(
      child: Text(
        place.name,
        style: TextStyle(
          fontSize: 14,
          color: Colors.grey.shade700,
        ),
      ),
    ),
    Text(
      place.entryFee != null && place.entryFee! > 0
        ? '₹${entryFee.toStringAsFixed(0)}'
        : 'FREE',
      style: TextStyle(
        fontSize: 14,
        fontWeight: FontWeight.w600,
        color: Colors.blue.shade700,
      ),
    ),
  ],
),
);
})),
],
),
),
),
),
const SizedBox(height: 16),

// Cost Summary
Card(
  color: Colors.blue.shade50,
  child: Padding(
    padding: const EdgeInsets.all(16),
    child: Column(
      children: [
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            const Text(
              'Total Cost',
              style: TextStyle(
                fontSize: 20,
                fontWeight: FontWeight.bold,
```

```

    ),
  ),
  Text(
    '₹${totalCost.toStringAsFixed(0)}',
    style: TextStyle(
      fontSize: 24,
      fontWeight: FontWeight.bold,
      color: Colors.blue.shade700,
    ),
  ),
],
),
const SizedBox(height: 8),
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  children: [
    const Text(
      'Budget',
      style: TextStyle(fontSize: 16),
    ),
    Text(
      '₹${budget.toStringAsFixed(0)}',
      style: const TextStyle(fontSize: 16),
    ),
  ],
),
const Divider(),
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  children: [
    Text(
      isWithinBudget ? 'Remaining' : 'Exceeded by',
      style: TextStyle(
        fontSize: 16,
        fontWeight: FontWeight.w600,
        color: isWithinBudget ? Colors.green : Colors.red,
      ),
    ),
    Text(
      '₹${(budget - totalCost).abs().toStringAsFixed(0)}',
      style: TextStyle(
        fontSize: 18,
        fontWeight: FontWeight.bold,
        color: isWithinBudget ? Colors.green : Colors.red,
      ),
    ),
  ],
),
],
),
),

```

```

    ),
  ),

  const SizedBox(height: 24),

  // Proceed to Payment button
  SizedBox(
    width: double.infinity,
    child: ElevatedButton(
      onPressed: () async {
        // Create booking
        final bookingService = BookingService();
        final booking = await bookingService.createBooking(
          userId: 'user_123', // In production, get from auth
          fromCity: tripProvider.fromCity!.name,
          toCity: tripProvider.toCity!.name,
          tripDate: tripProvider.tripDate ?? DateTime.now(),
          numberOfTravelers: tripProvider.numberOfTravelers,
          numberOfDays: tripProvider.numberOfDays,
          selectedTransport: tripProvider.selectedTransport,
          selectedHotel: tripProvider.selectedHotel,
          selectedPlaces: tripProvider.selectedPlaces,
          selectedRestaurants: tripProvider.selectedRestaurants,
          totalCost: totalCost,
          budget: budget,
        );

        Navigator.push(
          context,
          MaterialPageRoute(
            builder: (context) => PaymentPage(booking: booking),
          ),
        );
      },
      style: ElevatedButton.styleFrom(
        backgroundColor: Colors.blue.shade700,
        foregroundColor: Colors.white,
        padding: const EdgeInsets.symmetric(vertical: 16),
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(12),
        ),
      ),
      child: const Text(
        'Proceed to Payment',
        style: TextStyle(
          fontSize: 18,
          fontWeight: FontWeight.bold,
        ),
      ),
    ),
  ),

```

```

    ),
  ),
],
),
);
},
),
);
}

Widget _buildSummaryRow(String label, String value, IconData icon) {
  return Padding(
    padding: const EdgeInsets.only(bottom: 12),
    child: Row(
      children: [
        Icon(icon, size: 20, color: Colors.grey.shade600),
        const SizedBox(width: 8),
        Text(
          '$label: ',
          style: TextStyle(
            fontSize: 16,
            color: Colors.grey.shade700,
          ),
        ),
        Text(
          value,
          style: const TextStyle(
            fontSize: 16,
            fontWeight: FontWeight.w600,
          ),
        ),
      ],
    ),
  );
}
}

```

Transport_page.dart:

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:intl/intl.dart';
import '../models/transport.dart';
import '../providers/trip_provider.dart';
import 'hotels_page.dart';

```

```

class TransportPage extends StatelessWidget {
  const TransportPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Select Transport'),
        backgroundColor: Colors.blue.shade700,
        foregroundColor: Colors.white,
      ),
      body: Consumer<TripProvider>(
        builder: (context, tripProvider, _) {
          if (tripProvider.isLoading) {
            return const Center(child: CircularProgressIndicator());
          }

          final transports = tripProvider.transportOptions;
          final selectedTransport = tripProvider.selectedTransport;

          return Column(
            children: [
              // Budget info
              Container(
                padding: const EdgeInsets.all(16),
                color: Colors.blue.shade50,
                child: Row(
                  mainAxisAlignment: MainAxisAlignment.spaceBetween,
                  children: [
                    Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: [
                        Text(
                          'Budget: ₹${tripProvider.budget.toStringAsFixed(0)}',
                          style: const TextStyle(
                            fontSize: 16,
                            fontWeight: FontWeight.bold,
                          ),
                        ),
                        Text(
                          'Travelers: ${tripProvider.numberOfTravelers}',
                          style: TextStyle(
                            fontSize: 14,
                            color: Colors.grey.shade700,
                          ),
                        ),
                      ],
                    ),
                  ],
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}

```

```

        ),
        if (selectedTransport != null)
            Text(
                'Selected: ${selectedTransport.type.emoji}
                ${selectedTransport.type.displayName}',
                style: TextStyle(
                    fontSize: 14,
                    color: Colors.blue.shade700,
                    fontWeight: FontWeight.w600,
                ),
            ),
    ],
),
),
),

// Transport list
Expanded(
    child: ListView.builder(
        padding: const EdgeInsets.all(16),
        itemCount: transports.length,
        itemBuilder: (context, index) {
            final transport = transports[index];
            final isSelected = selectedTransport?.id == transport.id;
            final totalPrice = transport.getPriceForPassengers(
                tripProvider.numberOfTravelers,
            );

            return Card(
                margin: const EdgeInsets.only(bottom: 12),
                elevation: isSelected ? 4 : 1,
                shape: RoundedRectangleBorder(
                    borderRadius: BorderRadius.circular(12),
                    side: BorderSide(
                        color: isSelected
                            ? Colors.blue.shade700
                            : Colors.transparent,
                        width: 2,
                    ),
                ),
                child: InkWell(
                    onTap: () {
                        tripProvider.setSelectedTransport(transport);
                    },
                    borderRadius: BorderRadius.circular(12),
                    child: Padding(
                        padding: const EdgeInsets.all(16),
                        child: Column(
                            crossAxisAlignment: CrossAxisAlignment.start,
                            children: [

```

```

Row(
  children: [
    Text(
      transport.type.emoji,
      style: const TextStyle(fontSize: 32),
    ),
    const SizedBox(width: 12),
    Expanded(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            transport.type.displayName,
            style: const TextStyle(
              fontSize: 18,
              fontWeight: FontWeight.bold,
            ),
          ),
          if (transport.operatorName != null)
            Text(
              transport.operatorName!,
              style: TextStyle(
                fontSize: 14,
                color: Colors.grey.shade600,
              ),
            ),
        ],
      ),
    ),
    Column(
      crossAxisAlignment: CrossAxisAlignment.end,
      children: [
        Text(
          '₹${totalPrice.toStringAsFixed(0)}',
          style: TextStyle(
            fontSize: 20,
            fontWeight: FontWeight.bold,
            color: Colors.blue.shade700,
          ),
        ),
        Text(
          'per person:
₹${transport.price.toStringAsFixed(0)}',
          style: TextStyle(
            fontSize: 12,
            color: Colors.grey.shade600,
          ),
        ),
      ],
    ),
  ],
),

```

```

    ],
  ),
  const SizedBox(height: 12),
  Row(
    children: [
      Icon(Icons.access_time, size: 16, color:
Colors.grey.shade600),

      const SizedBox(width: 4),
      Text(
        '${transport.durationHours} hours',
        style: TextStyle(
          fontSize: 14,
          color: Colors.grey.shade600,
        ),
      ),
    ],
    const SizedBox(width: 16),
    Icon(Icons.calendar_today, size: 16, color:
Colors.grey.shade600),

    const SizedBox(width: 4),
    Text(
      DateFormat('MMM dd, hh:mm
a').format(transport.departureTime),
      style: TextStyle(
        fontSize: 14,
        color: Colors.grey.shade600,
      ),
    ),
  ],
),
if (transport.rating != null) ...[
  const SizedBox(height: 8),
  Row(
    children: [
      Icon(Icons.star, size: 16, color: Colors.amber),
      const SizedBox(width: 4),
      Text(
        transport.rating!.toStringAsFixed(1),
        style: const TextStyle(fontWeight:
FontWeight.w600),

      ),
      const SizedBox(width: 8),
      Text(
        '(${transport.availableSeats} seats available)',
        style: TextStyle(
          fontSize: 12,
          color: Colors.grey.shade600,
        ),
      ),
    ],
  ),
],
),

```

```

    ],
    if (isSelected) ...[
      const SizedBox(height: 8),
      Container(
        padding: const EdgeInsets.symmetric(
          horizontal: 8,
          vertical: 4,
        ),
        decoration: BoxDecoration(
          color: Colors.blue.shade700,
          borderRadius: BorderRadius.circular(4),
        ),
        child: const Text(
          'SELECTED',
          style: TextStyle(
            color: Colors.white,
            fontSize: 12,
            fontWeight: FontWeight.bold,
          ),
        ),
      ),
    ],
  ],
),
);
},
),
),
);

// Continue button
Container(
  padding: const EdgeInsets.all(16),
  decoration: BoxDecoration(
    color: Colors.white,
    boxShadow: [
      BoxShadow(
        color: Colors.black.withOpacity(0.1),
        blurRadius: 4,
        offset: const Offset(0, -2),
      ),
    ],
  ),
  child: SafeArea(
    child: ElevatedButton(
      onPressed: selectedTransport == null
        ? null
        : () {

```

```

        Navigator.push(
          context,
          MaterialPageRoute(
            builder: (context) => const HotelsPage(),
          ),
        );
      },
    style: ElevatedButton.styleFrom(
      backgroundColor: Colors.blue.shade700,
      foregroundColor: Colors.white,
      padding: const EdgeInsets.symmetric(vertical: 16),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12),
      ),
    ),
    child: const Text(
      'Continue to Hotels',
      style: TextStyle(
        fontSize: 18,
        fontWeight: FontWeight.bold,
      ),
    ),
  ),
),
),
),
),
],
);
},
),
);
}
}
}

```

Main.dart:

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'providers/trip_provider.dart';
import 'pages/home_page.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {

```

```

const MyApp({super.key});

@override
Widget build(BuildContext context) {
  return ChangeNotifierProvider(
    create: (_) => TripProvider(),
    child: MaterialApp(
      title: 'Pocket Trip',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.blue,
          primary: Colors.blue.shade700,
        ),
        useMaterial3: true,
        appBarTheme: AppBarTheme(
          backgroundColor: Colors.blue.shade700,
          foregroundColor: Colors.white,
          elevation: 0,
        ),
        cardTheme: CardThemeData(
          elevation: 2,
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(12),
          ),
        ),
        elevatedButtonTheme: ElevatedButtonThemeData(
          style: ElevatedButton.styleFrom(
            backgroundColor: Colors.blue.shade700,
            foregroundColor: Colors.white,
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(12),
            ),
          ),
        ),
      ),
      home: const HomePage(),
    ),
  );
}

```

Backend:

```
require('dotenv').config();

const express = require("express");
const mongoose = require("mongoose");
const axios = require("axios");
const cors = require("cors");


const app = express();
app.use(cors());
app.use(express.json());


mongoose.connect("mongodb+srv://pockettrip:pocket123@pockettripcluster.gv8lcms.mongodb.net/pockettripDB?retryWrites=true&w=majority").then(function()
{ console.log("MongoDB Connected"); }).catch(function(err) { console.log("Error:", err); });


const placeSchema = new mongoose.Schema({
  xid: String,
  name: String,
  type: String,
  kinds: String,
  address: String,
  rating: Number,
  lat: Number,
  lon: Number,
  city: String,
  image: String,
  description: String
```

```
}, { collection: "places" });
```

```
const Place = mongoose.model("Place", placeSchema);
```

```
const OTM_KEY = process.env.OTM_API_KEY;
```

```
const OTM_BASE = "https://api.opentripmap.com/0.1/en/places";
```

```
const GOOGLE_MAPS_API_KEY = process.env.GOOGLE_MAPS_API_KEY;
```

```
const GOOGLE_BASE = "https://maps.googleapis.com/maps/api/place";
```

```
const HOTELS_API_KEY = process.env.HOTELS_API_KEY;
```

```
const HOTELS_API_HOST = process.env.HOTELS_API_HOST;
```

```
const cityCoords = {
```

```
  hyderabad:    { lat: 17.3850, lon: 78.4867 },
```

```
  mumbai:       { lat: 19.0760, lon: 72.8777 },
```

```
  delhi:        { lat: 28.6139, lon: 77.2090 },
```

```
  bangalore:    { lat: 12.9716, lon: 77.5946 },
```

```
  chennai:      { lat: 13.0827, lon: 80.2707 },
```

```
  kolkata:      { lat: 22.5726, lon: 88.3639 },
```

```
  tirupati:     { lat: 13.6288, lon: 79.4192 },
```

```
  varanasi:     { lat: 25.3176, lon: 82.9739 },
```

```
  agra:         { lat: 27.1767, lon: 78.0081 },
```

```
  jaipur:       { lat: 26.9124, lon: 75.7873 },
```

```
  goa:          { lat: 15.2993, lon: 74.1240 },
```

```
  mysore:       { lat: 12.2958, lon: 76.6394 },
```

```
  amritsar:     { lat: 31.6340, lon: 74.8723 },
```

```
  rajamahendravaram: { lat: 17.0005, lon: 81.8040 },
```

```
visakhapatnam: { lat: 17.6868, lon: 83.2185 },  
vijayawada:    { lat: 16.5062, lon: 80.6480 }  
};
```

```
async function fetchPlaces(city, kinds, limit) {  
  const coords = cityCoords[city.toLowerCase()];  
  if (!coords) throw new Error("City not found");
```

```
  console.log("Fetching places for city:", city, "with kinds:", kinds);
```

```
  const listRes = await axios.get(OTM_BASE + "/radius", {  
    params: { radius: 15000, lon: coords.lon, lat: coords.lat, kinds: kinds, limit: limit, apikey:  
OTM_KEY }  
  });
```

```
  console.log("API Response status:", listRes.status, "Features count:",  
listRes.data.features?.length || 0);
```

```
  const features = listRes.data.features || [];
```

```
  const detailed = await Promise.all(  
    features.slice(0, 10).map(async function(f) {  
      try {  
        const det = await axios.get(OTM_BASE + "/xid/" + f.properties.xid, { params: { apikey:  
OTM_KEY } });  
        const d = det.data;  
        const place = {  
          xid: d.xid,
```

```

        name: d.name || "Unknown",
        kinds: d.kinds,
        address: d.address ? (d.address.road || "") + " " + (d.address.city || "") : "",
        rating: d.rate || 0,
        lat: d.point ? d.point.lat : null,
        lon: d.point ? d.point.lon : null,
        city: city,
        image: d.preview ? d.preview.source : null,
        description: d.wikipedia_extracts ? d.wikipedia_extracts.text.slice(0, 300) : "",
        source: "opentripmap"
    };

    console.log("Fetched place:", place.name);

    return place;
} catch(e) {
    console.log("Error fetching place details:", e.message);
    return null;
}
})
);

const filtered = detailed.filter(Boolean);
console.log("Total places after filtering:", filtered.length);
return filtered;
}

// Save places to database

```

```
async function savePlacesToDB(places) {  
  try {  
    console.log("Saving", places.length, "places to database...");  
    let count = 0;  
  
    for (const place of places) {  
      if (!place) continue;  
  
      const query = place.xid ? { xid: place.xid } : { place_id: place.place_id };  
      console.log("Query:", query);  
  
      const result = await Place.findOneAndUpdate(  
        query,  
        { $set: place },  
        { upsert: true, new: true }  
      );  
  
      if (result) {  
        count++;  
        console.log("✅ Saved place:", place.name);  
      }  
    }  
  
    console.log("✅ Total places saved to DB:", count);  
    return count;  
  } catch (error) {
```

```
console.log(" ❌ Error saving places to DB:", error.message);

console.log("Error stack:", error);

return 0;

}

}

// Google Places API - Fetch Historical Places

async function fetchHistoricalPlacesGoogle(city) {

  const coords = cityCoords[city.toLowerCase()];

  if (!coords) throw new Error("City not found");

  try {

    if (!GOOGLE_MAPS_API_KEY) throw new Error("Google Maps API key not configured");

    const searchQuery = city + " historical places monuments temples";

    const response = await
    axios.get("https://maps.googleapis.com/maps/api/place/textsearch/json", {

      params: {

        query: searchQuery,

        location: coords.lat + "," + coords.lon,

        radius: 15000,

        type: "point_of_interest",

        key: GOOGLE_MAPS_API_KEY

      }

    });
```

```

const places = await Promise.all(
  (response.data.results || []).slice(0, 15).map(async function(place) {
    try {
      const detailsResponse = await
        axios.get("https://maps.googleapis.com/maps/api/place/details/json", {
          params: {
            place_id: place.place_id,
            fields:
              "formatted_address,rating,types,geometry,photos,reviews,opening_hours,url",
            key: GOOGLE_MAPS_API_KEY
          }
        });

      const details = detailsResponse.data.result;

      return {
        name: place.name,
        place_id: place.place_id,
        address: details.formatted_address || place.formatted_address || "",
        rating: details.rating || place.rating || 0,
        lat: place.geometry.location.lat,
        lon: place.geometry.location.lng,
        city: city,
        types: place.types ? place.types.join(",") : "",
        source: "google_places",
        image: place.photos && place.photos[0] ? place.photos[0].photo_reference : null,
        url: details.url || ""
      };
    } catch (error) {
      console.log(error);
    }
  })
);

```

```
        opening_hours: details.opening_hours ? details.opening_hours.weekday_text : []
    };
} catch(e) {
    console.log("Error fetching place details:", e.message);
    return null;
}
})
);
```

```
    return places.filter(Boolean);
} catch(error) {
    console.log("Google Places API Error:", error.message);
    throw error;
}
}
```

// Hotels API - Search and Get Hotel Details

```
async function fetchHotels(searchQuery, checkIn, checkOut, adults = 1) {
    try {
        if (!HOTELS_API_KEY) throw new Error("Hotels API Key not configured");

        const options = {
            method: "GET",
            url: "https://hotels4.p.rapidapi.com/properties/get-details",
            params: {
                id: searchQuery,
```

```
    checkIn: checkIn,  
    checkOut: checkOut,  
    adults1: adults,  
    currency: "USD",  
    locale: "en_US"  
  },  
  headers: {  
    "x-rapidapi-key": HOTELS_API_KEY,  
    "x-rapidapi-host": HOTELS_API_HOST  
  }  
};
```

```
const response = await axios.request(options);
```

```
const hotel = response.data;
```

```
return {  
  id: hotel.id || searchQuery,  
  name: hotel.name || "Unknown Hotel",  
  description: hotel.description || "",  
  rating: hotel.reviews && hotel.reviews.score ? hotel.reviews.score : 0,  
  address: hotel.address ? hotel.address.address1 + ", " + (hotel.address.city || "") : "",  
  city: hotel.address ? hotel.address.city : "",  
  lat: hotel.coordinates ? hotel.coordinates.latitude : null,  
  lon: hotel.coordinates ? hotel.coordinates.longitude : null,  
  checkIn: checkIn,  
  checkOut: checkOut,
```

```

    adults: adults,
    price: hotel.price ? hotel.price.current : "Not available",
    currency: "USD",
    amenities: hotel.amenities || [],
    images: hotel.images || [],
    source: "hotels4_api"
  };
} catch(error) {
  console.log("Hotels API Error:", error.message);
  throw error;
}
}

app.get("/v1", function(req, res) {
  res.send("PocketTrip Backend Running");
});

app.get("/cities", function(req, res) {
  res.json({ cities: Object.keys(cityCoords) });
});

app.get("/places/tourist", async function(req, res) {
  try {
    const city = req.query.city;
    if (!city) return res.status(400).json({ error: "city is required" });
    const places = await fetchPlaces(city, "tourist_facilities,natural,historic", 20);
  }
});

```

```
    const saved = await savePlacesToDB(places);
    res.json({ city: city, count: places.length, saved: saved, places: places });
  } catch(error) {
    res.status(500).json({ error: error.message });
  }
});
```

```
app.get("/places/temples", async function(req, res) {
  try {
    const city = req.query.city;
    if (!city) return res.status(400).json({ error: "city is required" });
    const places = await fetchPlaces(city, "religion,hindu_temples", 20);
    const saved = await savePlacesToDB(places);
    res.json({ city: city, count: places.length, saved: saved, places: places });
  } catch(error) {
    res.status(500).json({ error: error.message });
  }
});
```

```
app.get("/places/historical", async function(req, res) {
  try {
    const city = req.query.city;
    if (!city) return res.status(400).json({ error: "city is required" });
    const places = await fetchPlaces(city, "historic,fortifications,archaeology", 20);
    const saved = await savePlacesToDB(places);
    res.json({ city: city, count: places.length, saved: saved, places: places });
  }
});
```

```
    } catch(error) {  
      res.status(500).json({ error: error.message });  
    }  
  });
```

```
app.get("/places/historical/google", async function(req, res) {  
  try {  
    const city = req.query.city;  
    if (!city) return res.status(400).json({ error: "city is required" });  
    if (!GOOGLE_MAPS_API_KEY || GOOGLE_MAPS_API_KEY ===  
"YOUR_GOOGLE_MAPS_API_KEY_HERE") {  
      return res.status(500).json({ error: "Google Maps API key not configured" });  
    }  
  
    const places = await fetchHistoricalPlacesGoogle(city);  
    const saved = await savePlacesToDB(places);  
    res.json({ city: city, count: places.length, saved: saved, source: "google_places", places:  
places });  
  } catch(error) {  
    res.status(500).json({ error: error.message });  
  }  
});
```

```
app.get("/places/family", async function(req, res) {  
  try {  
    const city = req.query.city;
```

```

    if (!city) return res.status(400).json({ error: "city is required" });

    const places = await fetchPlaces(city,
    "amusements,zoos,water_parks,gardens_and_parks", 20);

    await savePlacesToDB(places);

    res.json({ city: city, count: places.length, saved: true, places: places });
  } catch(error) {

    res.status(500).json({ error: error.message });

  }
});

```

```

app.get("/places/restaurants", async function(req, res) {

  try {

    const city = req.query.city;

    if (!city) return res.status(400).json({ error: "city is required" });

    const coords = cityCoords[city.toLowerCase()];

    if (!coords) return res.status(400).json({ error: "City not found" });

    const query = '[out:json];node["amenity"]="restaurant"(around:5000,' + coords.lat + ',' +
    coords.lon + ');out 20;';

    const response = await axios.post("https://overpass-api.de/api/interpreter", query,
    { headers: { "Content-Type": "text/plain" } });

    const restaurants = response.data.elements.map(function(el) {

      return {

        name: el.tags && el.tags.name ? el.tags.name : "Unknown",

        cuisine: el.tags && el.tags.cuisine ? el.tags.cuisine : "Not specified",

        lat: el.lat,

```

```
    lon: el.lon
  };
});

res.json({ city: city, count: restaurants.length, restaurants: restaurants });
} catch(error) {
  res.status(500).json({ error: error.message });
}
});

app.get("/weather", async function(req, res) {
  try {
    const city = req.query.city;
    if (!city) return res.status(400).json({ error: "city is required" });
    const coords = cityCoords[city.toLowerCase()];
    if (!coords) return res.status(400).json({ error: "City not found" });

    const response = await axios.get("https://api.open-meteo.com/v1/forecast", {
      params: { latitude: coords.lat, longitude: coords.lon, current_weather: true }
    });

    const w = response.data.current_weather;

    res.json({ city: city, temperature: w.temperature + "C", windspeed: w.windspeed + "
km/h" });
  } catch(error) {
    res.status(500).json({ error: error.message });
  }
});
```

```
}  
});
```

```
app.get("/hotels/details", async function(req, res) {  
  try {  
    const hotelId = req.query.id;  
    const checkIn = req.query.checkIn;  
    const checkOut = req.query.checkOut;  
    const adults = req.query.adults || 1;  
  
    if (!hotelId) return res.status(400).json({ error: "hotel id is required" });  
    if (!checkIn) return res.status(400).json({ error: "checkIn date is required (YYYY-MM-DD)" });  
    if (!checkOut) return res.status(400).json({ error: "checkOut date is required (YYYY-MM-DD)" });  
  
    if (!HOTELS_API_KEY) return res.status(500).json({ error: "Hotels API key not configured" });  
  
    const hotelData = await fetchHotels(hotelId, checkIn, checkOut, adults);  
    res.json({ hotel: hotelData });  
  } catch(error) {  
    res.status(500).json({ error: error.message });  
  }  
});
```

```
app.post("/places/save", async function(req, res) {
```

```
try {  
  const place = new Place(req.body);  
  await place.save();  
  res.json({ message: "Place saved!", place: place });  
} catch(error) {  
  res.status(500).json({ error: error.message });  
}  
});
```

```
app.get("/places/saved", async function(req, res) {  
  try {  
    const places = await Place.find({});  
    res.json({ count: places.length, places: places });  
  } catch(error) {  
    res.status(500).json({ error: error.message });  
  }  
});
```

```
const PORT = process.env.PORT || 3001;  
app.listen(PORT, function() {  
  console.log("Server running on port " + PORT);  
});
```

